

Guia Rápido: Setup LangChain + LangGraph

1. Ambiente Virtual

```
# Criar
python3 -m venv .venv
source .venv/bin/activate    # macOS/Linux
# .venv\Scripts\activate    # Windows

# Confirmar
which python    # deve mostrar .venv/bin/python
python --version
```

2. Instalar Dependências

```
# Core
pip install langgraph langchain-core langchain-openai

# Úteis
pip install httpx python-dotenv aiohttp

# Tudo de uma vez
pip install langgraph langchain-core langchain-openai httpx
python-dotenv aiohttp
```

3. Configurar API Key

```
# Criar .env
echo 'OPENROUTER_API_KEY=sk-or-v1-sua-chave-aqui' > .env
```

Obtenha sua chave gratuita em: <https://openrouter.ai/keys>

4. Testar Conexão

```
# test.py
import os, httpx, json
from dotenv import load_dotenv
load_dotenv()
```

```

resp = httpx.post(
    "https://openrouter.ai/api/v1/chat/completions",
    headers={
        "Authorization": f"Bearer {os.getenv('OPENROUTER_API_KEY')}",
        "Content-Type": "application/json",
    },
    json={
        "model": "google/gemini-2.5-flash-lite",
        "messages": [{"role": "user", "content": "Diga olá em uma frase!"}],
    },
    timeout=30,
)
print(resp.json()["choices"][0]["message"]["content"])

python test.py
# Deve imprimir algo como: "Olá! Como posso ajudar você hoje?"

```

5. Editor (VS Code)

Extensões recomendadas: - **Python** (Microsoft) - **Pylance** - **Even Better TOML** (opcional para pyproject.toml)

Troubleshooting

Problema	Solução
pip: command not found	Instale Python 3.10+ de python.org
ModuleNotFoundError: langgraph	Ative o venv: source .venv/bin/activate
401 Unauthorized	Verifique OPENROUTER_API_KEY no .env
Timeout	Verifique conexão com internet

Próximo: [02-primeiro-projeto.md](#)

Guia Rápido: Primeiro Projeto com LangChain

1. Chamada Simples a um LLM

```
# 01_basico.py
import os, httpx
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")

def perguntar(prompt, modelo="google/gemini-2.5-flash-lite"):
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": modelo,
            "messages": [{"role": "user", "content": prompt}],
        },
        timeout=30,
    )
    return resp.json()["choices"][0]["message"]["content"]

# Teste
print(perguntar("Explique o que é LangChain em 1 frase."))
print(perguntar("Qual a capital do Brasil?", "openai/gpt-4o"))
```

2. Memória de Conversa

```
# 02_memoria.py
import os, httpx
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")
historico = []

def conversar(msg, modelo="google/gemini-2.5-flash-lite"):
    historico.append({"role": "user", "content": msg})
```

```

resp = httpx.post(
    "https://openrouter.ai/api/v1/chat/completions",
    headers={
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json",
    },
    json={
        "model": modelo,
        "messages": historico, # envia histórico todo
    },
    timeout=30,
).json()

resposta = resp["choices"][0]["message"]["content"]
historico.append({"role": "assistant", "content": resposta})
return resposta

print(conversar("Meu nome é Rodolfo, sou de Campinas."))
print(conversar("Qual meu nome e de onde eu sou?"))
# O modelo lembra porque o histórico está no array

```

3. System Prompt (Personalidade)

```

# 03_persona.py
import os, httpx
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")

SYSTEM = "Você é Jarvis, um assistente de IA estilo Homem de Ferro. Seja conciso e use português."

def jarvis(prompt):
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": "openai/gpt-4o",
            "messages": [
                {"role": "system", "content": SYSTEM},
                {"role": "user", "content": prompt},
            ],
        },
        timeout=30,
    ).json()
    return resp["choices"][0]["message"]["content"]

```

```
print(jarvis("Qual a temperatura hoje?"))
```

4. Tool Calling (Function Calling)

```
# 04_tools.py
import os, httpx, json
from datetime import datetime
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")

TOOLS = [{
    "type": "function",
    "function": {
        "name": "get_time",
        "description": "Retorna data e hora atual",
        "parameters": {"type": "object", "properties": {}},
        "required": []},
    },
]

def executar_ferramenta(nome, args):
    if nome == "get_time":
        return datetime.now().strftime("%H:%M:%S de %d/%m/%Y")
    return "Ferramenta não encontrada"

def perguntar_com_tools(prompt):
    messages = [{"role": "user", "content": prompt}]

    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": "google/gemini-2.5-flash-lite",
            "messages": messages,
            "tools": TOOLS,
            "tool_choice": "auto",
        },
        timeout=30,
    ).json()

    msg = resp["choices"][0]["message"]

    if msg.get("tool_calls"):
        for tc in msg["tool_calls"]:
            nome = tc["function"]["name"]
            args = json.loads(tc["function"]["arguments"])
```

```

        resultado = executar_ferramenta(nome, args)

        messages.append(msg)
        messages.append({
            "role": "tool",
            "tool_call_id": tc["id"],
            "content": resultado,
        })

# Segunda chamada com o resultado da tool
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": "google/gemini-2.5-flash-lite",
            "messages": messages,
        },
        timeout=30,
    ).json()
    return resp["choices"][0]["message"]["content"]

    return msg.get("content", "")

print(perguntar_com_tools("Que horas são agora?"))

```

5. LangChain: Prompt Templates

```

# 05_langchain_prompt.py
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é um {profissao} especialista em {area}."),
    ("user", "{pergunta}"),
])

template = prompt.invoke({
    "profissao": "copywriter",
    "area": "marketing digital",
    "pergunta": "Crie uma headline para um curso de IA",
})

print("System:", template.messages[0].content)
print("User:", template.messages[1].content)
# Envie para o LLM com o código dos exemplos anteriores

```

6. LangChain: Chains

```
# 06_chain.py
import os, httpx
from langchain_core.prompts import ChatPromptTemplate
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")
MODEL = "google/gemini-2.5-flash-lite"

prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é um especialista em {topico}. Responda em português."),
    ("user", "{pergunta}"),
])

def llm_call(messages):
    """Função que chama o LLM - adapte para seu provider"""
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {API_KEY}", "Content-Type": "application/json"},
        json={"model": MODEL, "messages": messages},
        timeout=30,
    ).json()
    return resp["choices"][0]["message"]["content"]

# Pipeline simples: prompt → LLM
pergunta = "O que é RAG em IA?"
topico = "inteligência artificial generativa"

messages = prompt.invoke({"topico": topico, "pergunta": pergunta})
mensagens_formatadas = [
    {"role": m.type if m.type != "system" else "system",
     "content": m.content}
    for m in messages.messages
]
resposta = llm_call(mensagens_formatadas)
print(resposta)
```

Modelos Disponíveis (OpenRouter)

Modelo	ID	Custo	Uso
Gemini 2.5 Flash Lite	google/gemini-2.5-flash-lite	Grátis	Tarefas simples
GPT-4o Mini	openai/gpt-4o-mini	Baixo	Rápido e bom
GPT-4o		Médio	Qualidade

Modelo	ID	Custo	Uso
	openai/ gpt-4o		
Claude Sonnet 4	anthropic/ claude- sonnet-4	Médio	Melhor qualidade
DeepSeek Chat	deepseek/ deepseek- chat	Grátis	Alternativa grátis

Próximo: [03-langgraph-intro.md](#)

Guia Rápido: Introdução ao LangGraph

Conceito Central

LangGraph = StateGraph onde cada nó é uma função Python e cada aresta é uma transição.

State (dict) → Node (função) → State atualizado → Node → ... → END

1. Primeiro Grafo: Eco

```
# 07_primeiro_grafo.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    mensagem: str
    status: str

def entrada(state: State) -> dict:
    """Nó que processa a entrada"""
    return {"status": "processado"}

def saida(state: State) -> dict:
    """Nó que prepara a saída"""
    return {"mensagem": f"Eco: {state['mensagem']}"}

# Montar grafo
grafo = StateGraph(State)
grafo.add_node("entrada", entrada)
grafo.add_node("saida", saida)
grafo.add_edge(START, "entrada")
grafo.add_edge("entrada", "saida")
grafo.add_edge("saida", END)

app = grafo.compile()

# Executar
resultado = app.invoke({"mensagem": "Olá LangGraph!"})
print(resultado)
# {'mensagem': 'Eco: Olá LangGraph!', 'status': 'processado'}
```

2. Conditional Edges (Roteamento)

```
# 08_rotas.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    texto: str
    sentimento: str
    resposta: str

def analisar_sentimento(state: State) -> dict:
    texto = state["texto"].lower()
    if any(p in texto for p in ["ótimo", "bom", "amo", "👍"]):
        return {"sentimento": "positivo"}
    elif any(p in texto for p in ["ruim", "péssimo", "odeio"]):
        return {"sentimento": "negativo"}
    return {"sentimento": "neutro"}

def responder_positivo(state: State) -> dict:
    return {"resposta": "Que bom! Fico feliz em ajudar 😊"}

def responder_negativo(state: State) -> dict:
    return {"resposta": "Sinto muito. Como posso melhorar?"}

def responder_neutro(state: State) -> dict:
    return {"resposta": "Entendi. Conte-me mais."}

def rotear(state: State) -> str:
    return state["sentimento"] # "positivo" | "negativo" | "neutro"

grafo = StateGraph(State)
grafo.add_node("analisar", analisar_sentimento)
grafo.add_node("positivo", responder_positivo)
grafo.add_node("negativo", responder_negativo)
grafo.add_node("neutro", responder_neutro)

grafo.add_edge(START, "analisar")
grafo.add_conditional_edges(
    "analisar",
    rotear,
    {
        "positivo": "positivo",
        "negativo": "negativo",
        "neutro": "neutro",
    },
)
grafo.add_edge("positivo", END)
grafo.add_edge("negativo", END)
grafo.add_edge("neutro", END)
```

```

app = grafo.compile()

print(app.invoke({"texto": "Que produto incrível!"}))
print(app.invoke({"texto": "Não gostei do atendimento"}))
print(app.invoke({"texto": "Qual o horário de funcionamento?"}))

```

3. Streaming (Eventos em Tempo Real)

```

# 09_streaming.py
import asyncio
from langgraph.graph import StateGraph, START, END

# Mesmo grafo do exemplo 2...
# (reuse a definição do grafo acima)

async def main():
    app = grafo.compile()

    async for event in app.astream(
        {"texto": "Que produto incrível!"},
        stream_mode="updates",
    ):
        for node_name, update in event.items():
            print(f"[{node_name}] → {update}")

asyncio.run(main())
# [analisar] → {'sentimento': 'positivo'}
# [positivo] → {'resposta': 'Que bom! Fico feliz em ajudar 😊'}

```

4. Checkpoints (Persistência)

```

# 10_checkpoints.py
from langgraph.checkpoint.memory import MemorySaver

# Ao compilar:
app = grafo.compile(checkpointer=MemorySaver())

config = {"configurable": {"thread_id": "conversa-123"}}

# Cada invoke com mesmo thread_id compartilha estado
r1 = app.invoke({"texto": "Meu nome é Rodolfo"}, config)
r2 = app.invoke({"texto": "Qual meu nome?"}, config)

# Histórico de checkpoints
estados = list(app.get_state_history(config))
for s in estados:
    print(s.values)

```

5. Send API (Fan-Out Paralelo)

```
# 11_fanout.py
from langgraph.types import Send

AGENTES = ["copywriter", "designer", "estrategista"]

def disparar_agentes(state: State) -> list[Send]:
    """Retorna Send para cada agente executar em paralelo"""
    return [
        Send("executar", {"agente": nome, "tarefa":
            state["texto"]})
        for nome in AGENTES
    ]

async def executar(state: State) -> dict:
    resultado = f"[{state['agente']}] executou: {state['tarefa']}
        [:50]}"
    return {"resultados": [resultado]}

grafo = StateGraph(State)
grafo.add_node("disparar", disparar_agentes)
grafo.add_node("executar", executar)
grafo.add_edge(START, "disparar")
grafo.add_edge("executar", END)

# Conditional edge do fanout:
# grafo.add_conditional_edges("disparar", disparar_agentes,
#                             path_map)
```

Fluxo Completo: Classificar → Responder

```
# 12_completo.py
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    route: str
    response: str

def classify(state: AgentState) -> dict:
    """Classifica a intenção"""
    msg = state["messages"][-1].content.lower()
    if "?" in msg:
        return {"route": "pergunta"}
    elif len(msg) > 200:
        return {"route": "analise"}
    return {"route": "rapido"}
```

```

def responder_pergunta(state: AgentState) -> dict:
    return {"response": "Resposta detalhada para sua pergunta..."}

def responder_analise(state: AgentState) -> dict:
    return {"response": "Análise profunda do seu texto..."}

def responder_rapido(state: AgentState) -> dict:
    return {"response": "Resposta rápida e direta."}

def rotear(state: AgentState) -> str:
    return state["route"]

grafo = StateGraph(AgentState)
grafo.add_node("classify", classify)
grafo.add_node("pergunta", responder_pergunta)
grafo.add_node("analise", responder_analise)
grafo.add_node("rapido", responder_rapido)

grafo.add_edge(START, "classify")
grafo.add_conditional_edges(
    "classify", rotear,
    {"pergunta": "pergunta", "analise": "analise", "rapido":
     "rapido"},
)
grafo.add_edge("pergunta", END)
grafo.add_edge("analise", END)
grafo.add_edge("rapido", END)

app = grafo.compile()

# Com mensagens LangChain
from langchain_core.messages import HumanMessage
result = app.invoke({"messages": [HumanMessage(content="O que é
    LangGraph?")]}))
print(result["response"])

```

Glossário Rápido

Termo	Significado
State	Dict tipado que carrega dados entre nós
Node	Função Python que processa o state
Edge	Conexão fixa entre dois nós
Conditional Edge	Roteamento dinâmico baseado no state
Send	Dispara múltiplas execuções paralelas
Checkpoint	Persiste o state após cada nó
Thread	Sessão isolada de conversa (thread_id)

Próximo: Volte ao [README do projeto](#) e execute o Mega Brain v2!